

Q1. Freestyle Job Build From Scratch

The screenshot shows a GitHub repository page for a user named 'DevRushi-engg' with the repository name 'mypython'. The repository is public and has 0 stars, 0 forks, and 0 watches. The repository contains three files: 'LICENSE', 'hello.py', and 'requirements.txt'. The 'README' tab is selected, showing a prompt to 'Add a README'. The right sidebar contains sections for 'About', 'Releases', 'Packages', and 'Contributors'.

Repository Details:

- Repository: mypython (Public)
- Owner: DevRushi-engg
- Stars: 0
- Forks: 0
- Watches: 0

Files:

File Name	Commit Message	Time Ago
LICENSE	Initial commit	14 minutes ago
hello.py	Add print statement for 'Hello Jenk...	13 minutes ago
requirements.txt	Create requirements.txt	8 minutes ago

README:

Add a README

Help people interested in this repository understand your project.

[Add a README](#)

About:

No description, website, or topics provided.

- MIT license
- Activity
- 0 stars
- 0 watching
- 0 forks

Releases:

No releases published

[Create a new release](#)

Packages:

No packages published

[Publish your first package](#)

Contributors: 1



Configure



General



Source Code Management



Triggers



Environment



Build Steps



Post-build Actions

General

Enabled

Description

Freestyle

Plain text [Preview](#)

☐ Discard old builds ?

☐ GitHub project

☐ This project is parameterized ?

☐ Throttle builds ?

☐ Execute concurrent builds if necessary ?

Advanced

Source Code Management

Connect and manage your code repository to automatically pull the latest code for your builds.

☐ None

☒ Git ?

Repositories ?

Repository URL ?

<https://github.com/DevRushi-engg/mypython.git>

Credentials ?

- none -

+ Add

Advanced

+ Add Repository

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

- ☐ Trigger builds remotely (e.g., from scripts) ?
- ☐ Build after other projects are built ?
- ☐ Build periodically ?
- ☐ GitHub hook trigger for GITScm polling ?
- ☒ Poll SCM ?

Schedule ?

H/5 * * * *

Would last have run at Thursday, September 10, 2026, 4:58:00 PM India Standard Time; would next run at Thursday, September 10, 2026, 5:03:00 PM India Standard Time.

- ☐ Ignore post-commit hooks ?

Build Steps

Automate your build process with ordered tasks like code compilation, testing, and deployment.

Execute shell ?

Command

See [the list of available environment variables](#)

```
python3 hello.py
```

Advanced ▾

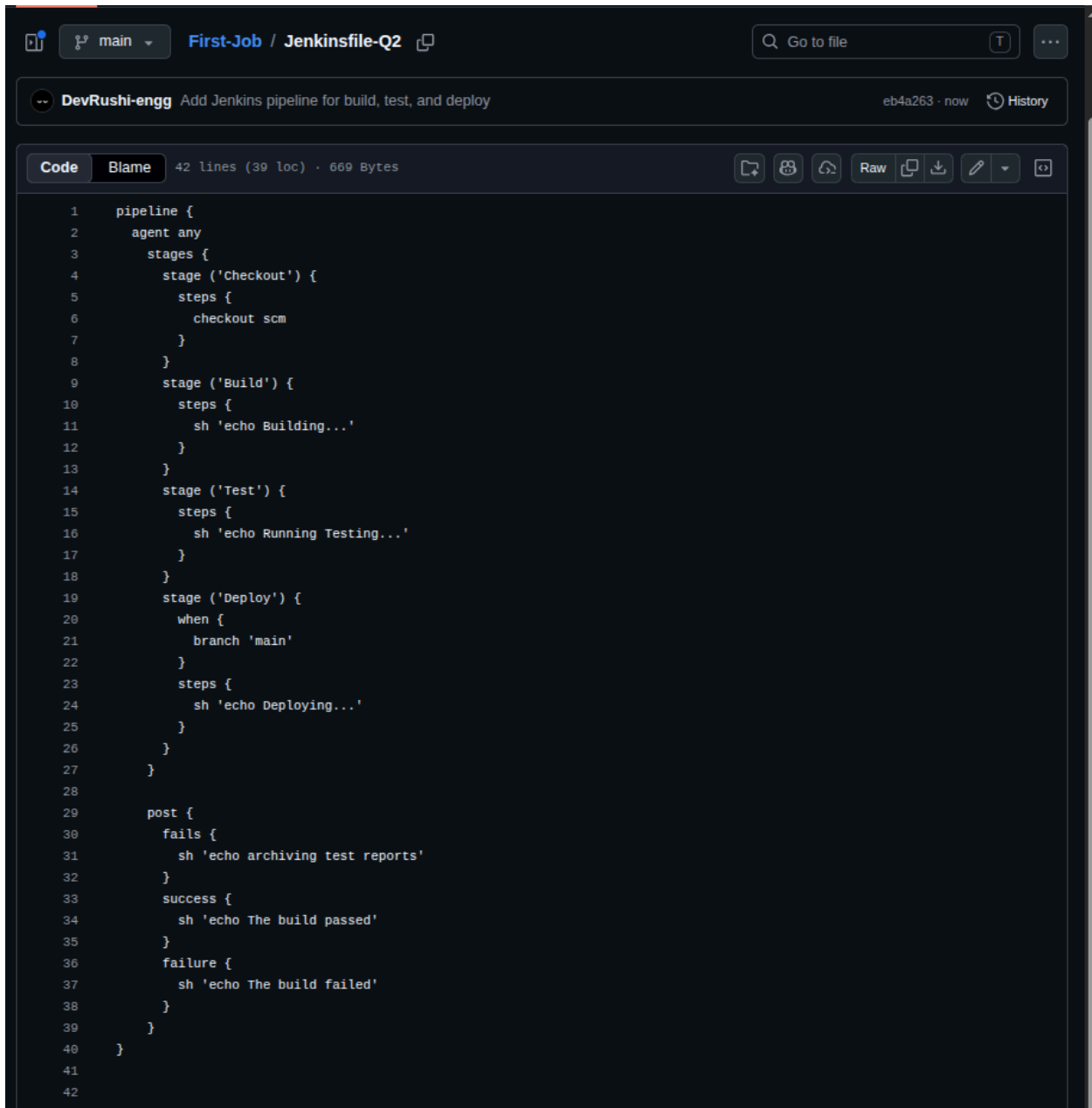
+ Add build step

Console

[Download](#)[Copy](#)[View as plain text](#)

```
Started by user Rushikesh
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/ci-demo-job
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/ci-demo-job/.git #
timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/DevRushi-engg/mypython.git #
timeout=10
Fetching upstream changes from https://github.com/DevRushi-engg/mypython.git
> git --version # timeout=10
> git --version # 'git version 2.39.2'
> git fetch --tags --force --progress -- https://github.com/DevRushi-
engg/mypython.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision fc2506c5e929a6ba10d8b27c2447608bd062996f
(refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f fc2506c5e929a6ba10d8b27c2447608bd062996f # timeout=10
Commit message: "Create requirements.txt"
> git rev-list --no-walk fc2506c5e929a6ba10d8b27c2447608bd062996f # timeout=10
[ci-demo-job] $ /bin/sh -xe /tmp/jenkins7365630280927829980.sh
+ python3 hello.py
Hello Jenkins
Finished: SUCCESS
```

Q2. Declarative Pipeline Conditional Multi-stage Build



```
1 pipeline {
2   agent any
3   stages {
4     stage ('Checkout') {
5       steps {
6         checkout scm
7       }
8     }
9     stage ('Build') {
10      steps {
11        sh 'echo Building...'
12      }
13    }
14    stage ('Test') {
15      steps {
16        sh 'echo Running Testing...'
17      }
18    }
19    stage ('Deploy') {
20      when {
21        branch 'main'
22      }
23      steps {
24        sh 'echo Deploying...'
25      }
26    }
27  }
28
29  post {
30    fails {
31      sh 'echo archiving test reports'
32    }
33    success {
34      sh 'echo The build passed'
35    }
36    failure {
37      sh 'echo The build failed'
38    }
39  }
40 }
41
42
```



< #2

Rerun



Started by Rushikesh

Started 4 min 56 sec ago

Took 19 sec



```

[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Checkout)
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/Pipess/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/DevRushi-engg/First-Job.git # timeout=10
Fetching upstream changes from https://github.com/DevRushi-engg/First-Job.git
> git --version # timeout=10
> git --version # 'git version 2.39.2'
> git fetch --tags --force --progress -- https://github.com/DevRushi-engg/First-Job.git
+refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision e027e208b80f9914b1f29ac81fa26b043749e2b9 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f e027e208b80f9914b1f29ac81fa26b043749e2b9 # timeout=10
Commit message: "Update Jenkinsfile-Q2"
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] sh
+ echo Building...
Building...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] sh
+ echo Running Testing...
Running Testing...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
Stage "Deploy" skipped due to when conditional
[Pipeline] getContext
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] sh
+ echo archiving test reports
archiving test reports
[Pipeline] sh
+ echo The build passed
The build passed

```

Q3.Debug the Broken Pipeline

Bug-1 -

The stage('Checkout') is outside the stages{ } block.

Bug-2 -

curly bracket was missing at stage ('Unit Tests')

Bug - 3

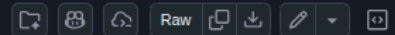
stage ('Parallel Tests') had a structural problem braces matching not complete the stage block

Bug - 4

the post must be outside the stages block it was inside the stage block.

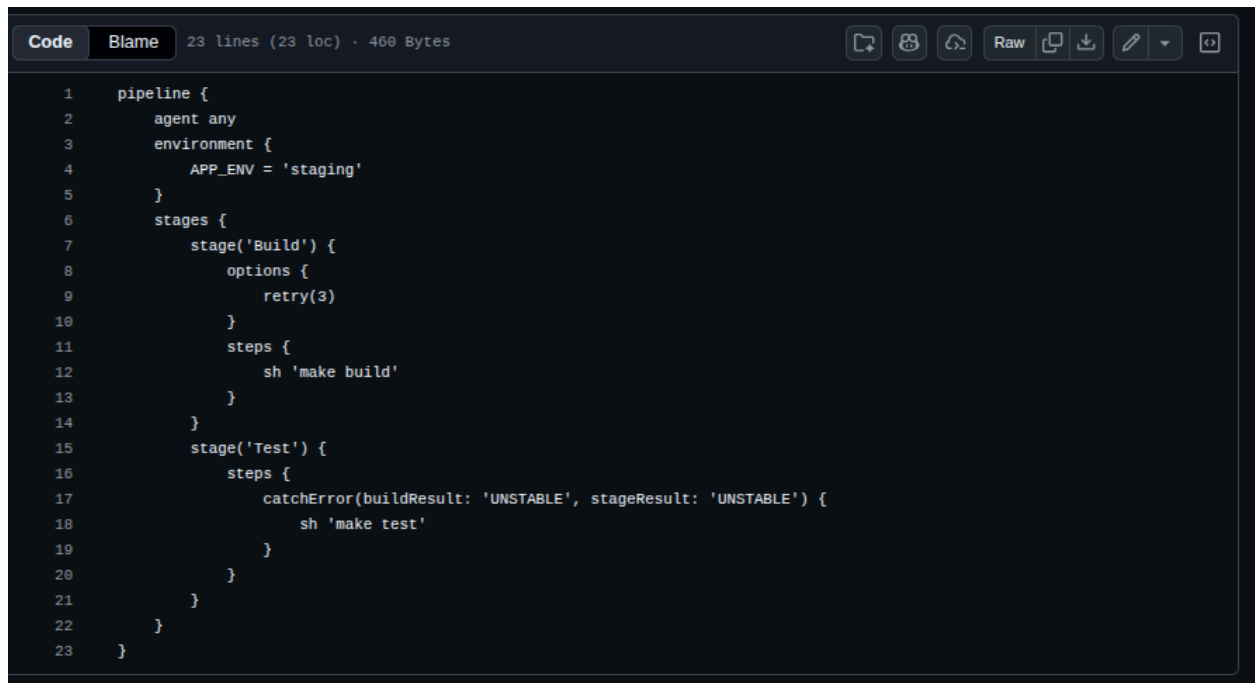
Fixed code:

Code Blame 41 lines (34 loc) · 1011 Bytes



```
1 pipeline {
2   agent any
3
4   stages {
5     stage ('Checkout') {           // Bug -1 : Moved the stage ('Checkout') inside Stages {}
6       steps {
7         git url: 'https://github.com/example/app.git'
8       }
9
10    stage ('Build') {
11      steps {
12        sh 'npm install'
13        sh 'npm test'
14      }
15    }
16
17    stage ('Parallel Tests') {
18      parallel {
19        stage ('Unit Tests') {      // Bug - 2 curly bracket was missing at stage ('Unit Tests')
20          steps {
21            sh 'npm test'
22          }
23        }
24
25        stage ('Lint') {
26          steps {
27            sh 'npm run lint'
28          }
29        }
30      }
31    }                                     // Bug - 3 stage ('Parallel Tests') had a structural problem braces matching not c
32
33    post {                             // Bug - 4 the post must be outside the stages block it was inside the stage block
34      always {
35        sh 'rm -rf workspace/**'
36      }
37    }
38  }
39 }
40
41
```

Q4.Scripted Vs Declarative fixed



```
1 pipeline {
2     agent any
3     environment {
4         APP_ENV = 'staging'
5     }
6     stages {
7         stage('Build') {
8             options {
9                 retry(3)
10            }
11            steps {
12                sh 'make build'
13            }
14        }
15        stage('Test') {
16            steps {
17                catchError(buildResult: 'UNSTABLE', stageResult: 'UNSTABLE') {
18                    sh 'make test'
19                }
20            }
21        }
22    }
23 }
```

Answer:

I would choose Scripted Pipeline because its more flexible for complex Groovy logic like loops. And declarative needs a script {} block for implementation.

Q5.Root Cause analysis

Symptoms - 1:

Cause:

**"Execute concurrent builds if necessary" is enabled with the custom workspace.
Due to which the "nightly-build is already locked" error is cause.**

Execute Concurrent build state that:

When this option is checked, multiple builds of this project may be executed in parallel. However, if you enable the *Use custom workspace* option, all builds will be executed in the same workspace. Therefore caution is required, as multiple builds may end up altering the same directory at the same time.

Fix:

There would be resource allocation problem may occur due to less resource available.

So, this need to be unchecked the **Execute concurrent builds if necessary**.

Symptoms - 2:

Cause:

Permission denied: cannot write to /var/jenkins_home/workspace/nightly-build/build.log

Fix:

The jenkins don't have ownership to the file in that workspace.

Symptoms - 3:

Cause:

Occasionally the build uses a startle copy of the repo missing the latest commit

Fix:

Checkout SCM is not used so used that in stage block to get complete source code from the repo.

Github webhooks can be use to get latest commit of the file.

Use Poll SCM to schedule the to scan the repo from github to get the latest commit.